

Assignment - 2

1. Create a constant array Prints

```
[[ 7. 7.]
 [ 7. 7.]]
```

```
import numpy as np
```

```
p = np.ones((2,2))*7
p
```

```
array([[7., 7.],
       [7., 7.]])
```

2. Create an array filled with random values.

```
Might print :[[ 0.91940167  0.08143941]
               [ 0.68744134  0.87236687]]
```

```
x = np.random.rand(2, 2)
x
```

```
array([[0.31923955, 0.01293614],
       [0.45159135, 0.96461959]])
```

3. Create a NumPy array containing the numbers from 10 to 50. Display:

```
The array,
Number of elements,
Shape,
Dimension,
Data type
```

```
a = np.arange(10, 51, 1)
print("Array a : ", a)
print("Length of a : ", len(a))
print("Shape of a : ", a.shape)
print("Dimension : ", a.ndim)
print("Data type of a : ", a.dtype)
```

```
Array a :  [10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33
 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50]
Length of a :  41
Shape of a :  (41,)
Dimension :  1
Data type of a :  int64
```

4. Using NumPy, create:

```
A 1D array containing 10 zeros
A 3 × 3 array containing ones
A 4 × 4 identity matrix
An array containing numbers from 5 to 50 with a step size of 5.
```

```
a = np.zeros(10)
print(" A 1D array containing 10 zeros : \n ", a)
b = np.ones((3, 3))
print(" A 3 × 3 array containing ones : \n", b)
print("A 4 × 4 identity matrix : \n", np.identity(4))
c = np.arange(5, 51, 5)
print("An array containing numbers from 5 to 50 with a step size of 5. : \n ", c)
```

```

A 1D array containing 10 zeros :
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
A 3 x 3 array containing ones :
[[1. 1. 1.]
 [1. 1. 1.]
 [1. 1. 1.]]
A 4 x 4 identity matrix :
[[1. 0. 0. 0.]
 [0. 1. 0. 0.]
 [0. 0. 1. 0.]
 [0. 0. 0. 1.]]
An array containing numbers from 5 to 50 with a step size of 5. :
[ 5 10 15 20 25 30 35 40 45 50]

```

5. Create an array containing numbers from 1 to 24. Reshape it into:

4 × 6

6 × 4

2 × 3 × 4

Display the shape and ndim of each result.

```

a = np.arange(1, 25, 1)
print("Array : ", a)
print("Shape : ", a.shape)
print("Dimension :", a.ndim)

b = a.reshape(4, 6)
print("Array(4 x 6) : ", b)
print("Shape : ", b.shape)
print("Dimension :", b.ndim)

c = a.reshape(6, 4)
print("Array(6 x 4) : ", c)
print("Shape : ", c.shape)
print("Dimension :", c.ndim)

d = a.reshape(2, 3, 4)
print("Array(2 x 3 x 4) : ", d)
print("Shape : ", d.shape)
print("Dimension :", d.ndim)

Array : [ 1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24]
Shape : (24,)
Dimension : 1
Array(4 x 6) : [[ 1  2  3  4  5  6]
 [ 7  8  9 10 11 12]
 [13 14 15 16 17 18]
 [19 20 21 22 23 24]]
Shape : (4, 6)
Dimension : 2
Array(6 x 4) : [[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]
Shape : (6, 4)
Dimension : 2
Array(2 x 3 x 4) : [[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
 [[13 14 15 16]
 [17 18 19 20]
 [21 22 23 24]]]
Shape : (2, 3, 4)
Dimension : 3

```

6. Consider: marks = np.array([78, 85, 92, 67, 88, 73, 95, 81])

Write NumPy statements to display:
First element

```
Last element
Third element
First four elements
Last three elements
```

```
marks = np.array([78, 85, 92, 67, 88, 73, 95, 81])
print("Marks : ", marks)

print("\nFirst element : ", marks[0])
print("Last element : ", marks[-1])
print("Third element : ", marks[2])
print("First four elements : ", marks[0:4])
print("Last three elements : ", marks[-3:])
```

```
Marks : [78 85 92 67 88 73 95 81]

First element : 78
Last element : 81
Third element : 92
First four elements : [78 85 92 67]
Last three elements : [73 95 81]
```

7. Create:

```
a = np.array([10, 20, 30, 40, 50])
b = np.array([5, 10, 15, 20, 25])
```

Calculate:

```
a + b
a - b
a * b
a / b
Square of every element in a
```

```
a = np.array([10, 20, 30, 40, 50])
b = np.array([5, 10, 15, 20, 25])

print("a : ", a)
print("b : ", b)

print("\na + b : ", a + b)
print("a - b : ", a - b)
print("a * b : ", a * b)
print("a / b : ", a / b)
print("Square of every element in a : ", a**2)
```

```
a : [10 20 30 40 50]
b : [ 5 10 15 20 25]

a + b : [15 30 45 60 75]
a - b : [ 5 10 15 20 25]
a * b : [ 50 200 450 800 1250]
a / b : [2. 2. 2. 2. 2.]
Square of every element in a : [ 100 400 900 1600 2500]
```

8. Given: salary = np.array([25000, 32000, 28000, 45000, 50000, 38000, 42000, 30000]) Find:

```
Total salary
Average salary
Highest salary
Lowest salary
Median salary
Standard deviation
```

```
salary = np.array([25000, 32000, 28000, 45000, 50000, 38000, 42000, 30000])
print("Salary : ", salary)
```

```
print("\nTotal Salary      : ", salary.sum())
print("Average Salary     : ", salary.mean())
print("Highest Salary     : ", salary.max())
print("Lowest Salary      : ", salary.min())
print("Median Salary      : ", np.median(salary))
print("Standard Deviation : ", salary.std())
```

```
Salary : [25000 32000 28000 45000 50000 38000 42000 30000]
```

```
Total Salary      : 290000
Average Salary     : 36250.0
Highest Salary     : 50000
Lowest Salary      : 25000
Median Salary      : 35000.0
Standard Deviation : 8317.902379807063
```

9. Given: marks = np.array([45, 78, 89, 32, 67, 90, 55, 76, 41, 84])

Using NumPy boolean indexing, find:

- Marks greater than 7
- Marks between 50 and 80
- Number of students who scored below 50
- Percentage of students who scored 60 or above
- Do not use a Python for loop.

```
marks = np.array([45, 78, 89, 32, 67, 90, 55, 76, 41, 84])
print("Maeks : ", marks)

print("\nMarks greater than 75      : ", marks[marks > 75])
print("Marks between 50 and 80      : ", marks[(marks > 50) & (marks < 80)])
print("Number of students who scored below 50 : ", np.sum(marks < 50))
print("Percentage of students who scored 60 or more : ", np.mean(marks >= 60)*100)
```

```
Maeks : [45 78 89 32 67 90 55 76 41 84]
```

```
Marks greater than 75      : [78 89 90 76 84]
Marks between 50 and 80    : [78 67 55 76]
Number of students who scored below 50 : 3
Percentage of students who scored 60 or more : 60.0
```

10. The following array contains marks of four students in three subjects:

```
marks = np.array([
```

```
[80, 75, 90],
```

```
[65, 70, 72],
```

```
[88, 92, 85],
```

```
[55, 60, 58]
```

```
])
```

Calculate:

- Total marks of each student
- Average marks of each student
- Average marks in each subject
- Student with the highest total
- Highest mark in each subject.

```
marks = np.array([[80, 75, 90],
                  [65, 70, 72],
                  [88, 92, 85],
                  [55, 60, 58]])

print("Total marks of each student : ")
```

```

print("Total marks of student 1 : ", np.sum(marks[0]))
print("Total marks of student 2 : ", np.sum(marks[1]))
print("Total marks of student 3 : ", np.sum(marks[2]))
print("Total marks of student 4 : ", np.sum(marks[3]))

print("\n\n Average marks of each student : ")

print("Average marks of student 1 : ", np.round(np.mean(marks[0]), 2))
print("Average marks of student 2 : ", np.round(np.mean(marks[1]), 2))
print("Average marks of student 3 : ", np.round(np.mean(marks[2]), 2))
print("Average marks of student 4 : ", np.round(np.mean(marks[3]), 2))

print("\n\nAverage marks in each subject : ")

subject_averages = np.mean(marks, axis=0)
print("Average marks in subject 1 : ", subject_averages[0])
print("Average marks in subject 2 : ", subject_averages[1])
print("Average marks in subject 3 : ", subject_averages[2])

total = np.sum(marks, axis=1)
highest_student_index = np.argmax(total)
print("\n\nStudent with the highest total : ")
print("Student number : ", highest_student_index + 1)
print("Highest total mark : ", total[highest_student_index])

subject_maximum = np.max(marks, axis=0)
print("\n\nHighest mark in each subject : ")
print("Highest mark in subject 1 : ", subject_maximum[0])
print("Highest mark in subject 2 : ", subject_maximum[1])
print("Highest mark in subject 3 : ", subject_maximum[2])

```

```

Total marks of each student :
Total marks of student 1 : 245
Total marks of student 2 : 207
Total marks of student 3 : 265
Total marks of student 4 : 173

```

```

Average marks of each student :
Average marks of student 1 : 81.67
Average marks of student 2 : 69.0
Average marks of student 3 : 88.33
Average marks of student 4 : 57.67

```

```

Average marks in each subject :
Average marks in subject 1 : 72.0
Average marks in subject 2 : 74.25
Average marks in subject 3 : 76.25

```

```

Student with the highest total :
Student number : 3
Highest total mark : 265

```

```

Highest mark in each subject :
Highest mark in subject 1 : 88
Highest mark in subject 2 : 92
Highest mark in subject 3 : 90

```

11. Create:

```
A = np.array([ [1, 2], [3, 4] ])
```

```
B = np.array([ [5, 6], [7, 8] ])
```

Perform:

```

Matrix addition
Element-wise multiplication
Matrix multiplication
Transpose of A.
explain the difference between: A * B and A @ B

```

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

print("Matrix addition : \n", A + B)
print("\nElement-wise multiplication : \n", A * B)
print("\nMatrix multiplication : \n", A @ B)
print("\nTranspose of A : \n", A.T)
```

```
Matrix addition :
[[ 6  8]
 [10 12]]
```

```
Element-wise multiplication :
[[ 5 12]
 [21 32]]
```

```
Matrix multiplication :
[[19 22]
 [43 50]]
```

```
Transpose of A :
[[1 3]
 [2 4]]
```

Difference between A*B and A@B:

- A * B performs element-wise multiplication, where numbers in the exact same positions are multiplied together.
- A @ B performs standard matrix multiplication, where rows of the first matrix are multiplied by the columns of the second matrix.

12. Daily temperatures for two weeks are:

```
temperature = np.array([ 31, 32, 33, 35, 36, 34, 32, 30, 29, 31, 33, 35, 37, 36 ])
```

Find:

```
Mean temperature
Maximum and minimum temperatures
Number of days above the average temperature
Temperatures greater than 34°C
Difference between maximum and minimum
Standard deviation
```

```
temperature = np.array([31, 32, 33, 35, 36, 34, 32, 30, 29, 31, 33, 35, 37, 36])
```

```
print("Mean temperature : ", np.round(np.mean(temperature), 2))
print("Number of days above the average temperature : ", np.sum(temperature > np.mean(temperature)))
print("Temperatures greater than 34°C : ", temperature[temperature > 34])
print("Difference between maximum and minimum : ", np.max(temperature) - np.min(temperature))
```

```
Mean temperature : 33.14
Number of days above the average temperature : 6
Temperatures greater than 34°C : [35 36 35 37 36]
Difference between maximum and minimum : 8
```

13. Monthly sales of a company are: sales = np.array([

```
12000, 15000, 17000, 14000,
18000, 21000, 19000, 22000,
25000, 23000, 26000, 28000
```

])

Determine:

```
Total annual sales
Average monthly sales
Month with the highest sales
Month with the lowest sales
```

Number of months with sales above ₹20,000
 Percentage increase from January to December

```
sales = np.array([12000, 15000, 17000, 14000, 18000, 21000, 19000, 22000, 25000, 23000, 26000, 28000])

print("Total annual sales : ", np.sum(sales))
print("Average monthly sales : ", np.round(np.mean(sales), 2))
print("Month with the highest sales : ", np.argmax(sales) + 1)
print("Month with the lowest sales : ", np.argmin(sales) + 1)
print("Number of months with sales above ₹20,000 : ", np.sum(sales > 20000))
print("Percentage increase from January to December : ", np.round((sales[-1] - sales[0]) / sales[0] * 100, 2))
```

```
Total annual sales : 240000
Average monthly sales : 20000.0
Month with the highest sales : 12
Month with the lowest sales : 1
Number of months with sales above ₹20,000 : 6
Percentage increase from January to December : 133.33
```

14. Consider marks of five students in four subjects:

```
marks = np.array([
```

```
[78, 85, 90, 88],
[65, 70, 72, 68],
[92, 88, 95, 91],
[55, 60, 58, 62],
[80, 75, 82, 79]
```

```
])
```

Write a NumPy program that calculates:

```
Total marks of every student
Average of every student
Average mark for every subject
Highest-scoring student
Lowest-scoring student
Students whose average is greater than 80
Overall class average
```

Then assign a performance category based on average:

Average	Category
≥ 85	Excellent
70–84.99	Good
50–69.99	Average
< 50	Needs Improvement

```
marks = np.array([
    [78, 85, 90, 88],
    [65, 70, 72, 68],
    [92, 88, 95, 91],
    [55, 60, 58, 62],
    [80, 75, 82, 79]
])

print("Total marks of every student : ", np.sum(marks, axis=1))
print("Average of every student : ", np.round(np.mean(marks, axis = 1), 2))
print("Average mark for every subject : ", np.round(np.mean(marks, axis = 0), 2))
print("Highest-scoring student : ", np.argmax(np.sum(marks, axis = 1)) + 1)
print("Lowest-scoring student : ", np.argmin(np.sum(marks, axis = 1)) + 1)
print("Students whose average is greater than 80 : ", np.sum(np.mean(marks, axis = 1) > 80))
print("Overall class average : ", np.round(np.mean(marks), 2))

print("\nPerformance Category(in Percentage) : ")
print("Excellent : ", np.mean(marks >= 85)* 100,"%")
print("Good : ", np.mean((marks >= 70) & (marks < 85))*100,"%")
print("Average : ", np.mean((marks >= 50) & (marks < 70))*100,"%")
print("Needs Improvement : ", np.mean(marks < 50)*100,"%")
```

```
Total marks of every student : [341 275 366 235 316]
Average of every student : [85.25 68.75 91.5  58.75 79.  ]
Average mark for every subject : [74.  75.6 79.4 77.6]
Highest-scoring student : 3
Lowest-scoring student : 4
Students whose average is greater than 80 : 2
Overall class average : 76.65

Performance Category(in Percentage) :
Excellent : 35.0 %
Good : 35.0 %
Average : 30.0 %
Needs Improvement : 0.0 %
```

15. Generate 20 random integers between 1 and 100.

```
import random
original_array = random.sample(range(1, 100), 20)
print(original_array)
```

```
[59, 40, 56, 24, 80, 53, 6, 76, 70, 43, 94, 61, 95, 31, 98, 92, 55, 32, 97, 85]
```

Find all even numbers from the generated array.

```
even_numbers = [x for x in original_array if x % 2 == 0]
print(even_numbers)
```

```
[40, 56, 24, 80, 6, 76, 70, 94, 98, 92, 32]
```

17. Replace values below 50 with 0.

```
original_array = np.array(original_array)
modified_array = np.where(original_array < 50, 0, original_array)
print("Modified Array ", modified_array)
```

```
Modified Array [59  0 56  0 80 53  0 76 70  0 94 61 95  0 98 92 55  0 97 85]
```

18. Sort the array in ascending and descending order.

```
ascending_array = np.sort(original_array)
print("Ascending array : ", ascending_array)

descending_array = np.sort(original_array)[::-1]
print("descending array : ", descending_array)
```

```
Ascending array : [ 6 24 31 32 40 43 53 55 56 59 61 70 76 80 85 92 94 95 97 98]
descending array : [98 97 95 94 92 85 80 76 70 61 59 56 55 53 43 40 32 31 24  6]
```

19. Find unique values and their frequencies.

```
unique_values, frequencies = np.unique(original_array, return_counts=True)

print("Unique Values:", unique_values)
print("Frequencies: ", frequencies)
```

```
Unique Values: [ 6 24 31 32 40 43 53 55 56 59 61 70 76 80 85 92 94 95 97 98]
Frequencies:   [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1]
```

20. Create a 5 × 5 random matrix and calculate row-wise and column-wise means.

```
matrix = np.random.randint(1, 101, size=(5, 5))
print("Original 5x5 Matrix:\n", matrix)

row_means = np.mean(matrix, axis=1)
print("\nRow-wise Means:\n", row_means)

col_means = np.mean(matrix, axis=0)
print("\nColumn-wise Means:\n", col_means)
```

```
Original 5x5 Matrix:
[[22 65 95  1 71]
 [29 16 10 57 84]
 [52 79 36 32 62]
 [39 65  1 51 19]
 [33 91 11  8 50]]
```

```
Row-wise Means:
[50.8 39.2 52.2 35.  38.6]
```

```
Column-wise Means:
[35.  63.2 30.6 29.8 57.2]
```

21. Take your own example to normalize an array.

```
original_array = np.array([55, 80, 45, 95, 60, 72, 88])
print("Original Array:", original_array)

min_val = np.min(original_array)
max_val = np.max(original_array)

normalized_array = (original_array - min_val) / (max_val - min_val)
print("Normalized Array (0 to 1):", np.round(normalized_array, 3))
```

```
Original Array: [55 80 45 95 60 72 88]
Normalized Array (0 to 1): [0.2  0.7  0.  1.  0.3  0.54 0.86]
```

22. use your own data to standardize an array.

```
prices = np.array([250, 410, 180, 520, 310, 290, 600, 380, 210, 450])
print("Original Prices Array:\n", prices)

mean_val = np.mean(prices)
std_val = np.std(prices)

standardized_prices = (prices - mean_val) / std_val

print("\nMean of original array: ", round(mean_val, 2))
print("Std Dev of original array:", round(std_val, 2))
print("\nStandardized Array (Z-scores):\n", np.round(standardized_prices, 3))

print("\nVerification:")
print("New Mean (should be 0): ", np.round(np.mean(standardized_prices)))
print("New Std Dev (should be 1): ", np.round(np.std(standardized_prices)))
```

```
Original Prices Array:
[250 410 180 520 310 290 600 380 210 450]
```

```
Mean of original array: 360.0
Std Dev of original array: 129.85
```

```
Standardized Array (Z-scores):
[-0.847  0.385 -1.386  1.232 -0.385 -0.539  1.848  0.154 -1.155  0.693]
```

```
Verification:
New Mean (should be 0):    -0.0
New Std Dev (should be 1):  1.0
```

23. Create a NumPy array containing 100 random exam marks and determine how many students fall into the ranges 0–39, 40–59, 60–79, and 80–100.

```
import numpy as np

marks = np.random.randint(0, 101, size=100)

conditions = [
    (marks >= 0) & (marks <= 39),
    (marks >= 40) & (marks <= 59),
    (marks >= 60) & (marks <= 79),
    (marks >= 80) & (marks <= 100)
]
choices = ["0-39", "40-59", "60-79", "80-100"]

categorized_marks = np.select(conditions, choices, default="Unknown")

ranges, counts = np.unique(categorized_marks, return_counts=True)

for r, c in zip(ranges, counts):
    print(f"Range {r}: {c} students")
```

```
Range 0-39: 46 students
Range 40-59: 16 students
Range 60-79: 19 students
Range 80-100: 19 students
```

24. Compare the time required to multiply 1,000,000 numbers using a Python list and a NumPy array.

```
import time
import numpy as np

size = 1000000
python_list = list(range(1, size + 1))
numpy_array = np.arange(1, size + 1)

start_time = time.time()
list_result = [x * 2 for x in python_list]
python_duration = time.time() - start_time
print(f"Python list execution time: {python_duration:.6f} seconds")

start_time = time.time()
array_result = numpy_array * 2
numpy_duration = time.time() - start_time
print(f"NumPy array execution time: {numpy_duration:.6f} seconds")

speedup = python_duration / numpy_duration
print(f"\nNumPy is approximately {speedup:.1f}x faster!")
```